

WealthWise

PERSONAL EXPENSE TRACKER

A Production-Ready Full-Stack Technical Project Report detailing the architecture, database schema, RESTful APIs, modern frontend, containerization, and continuous integration pipeline.

TECHNICAL PROJECT REPORT

Course: Software Engineering Mini Project Report

Technology Stack: FastAPI, SQLite, SQLAlchemy, Docker, GitHub Actions

Date: August 2026

Table of Contents

1. Introduction and Objectives	Page 3
2. Project Architecture & Technologies Used	Page 4
3. System Features & Functional Modules	Page 5
4. Database Schema and Data Models	Page 6
5. RESTful API Specification	Page 7
6. User Interface Design and Frontend Implementation	Page 8
7. Containerization & Deployment Configuration	Page 9
8. Continuous Integration & Pipeline Automation	Page 10
9. Summary & Future Scope	Page 11

1. Introduction and Objectives

In the contemporary digital era, personal financial management has transitioned from manual ledger books to interactive software applications. WealthWise is a lightweight, responsive, and robust Personal Expense Tracker designed to empower users to record, categorize, and analyze their expenditures efficiently. The application provides an immediate graphical and structural overview of spending habits, helping users identify primary expenditure categories and maintain strict budgets.

Project Objectives

- **Lightweight Architecture:** Build a lightweight client-server Single Page Application (SPA) that runs with minimal system configuration.
- **Data Persistence:** Ensure relational integrity and persistent transaction storage using a relational SQLite database schema.
- **Full CRUD Support:** Implement a complete suite of CRUD (Create, Read, Update, Delete) HTTP methods via a RESTful API backend.
- **Rich UI Aesthetics:** Design a premium, responsive glassmorphic UI using standard HTML5 and Vanilla CSS3 without external styling frameworks.
- **CI/CD Readiness:** Verify continuous code integration using automated linting, checking, and docker-build flows via GitHub Actions.
- **Production Portability:** Enable one-click containerized deployment using Docker and cloud hosting compatibility (e.g., Render.com).

2. Project Architecture & Technologies Used

WealthWise employs a decoupled client-server architecture model. The client (frontend) communicating over standard HTTP protocols with the server (FastAPI backend). This ensures rapid data rendering, low server overhead, and clear separation of concerns.

Technology Stack Overview: Below is the comprehensive list of core technology stacks utilized in this project:

Component Layer	Technology Selected	Technical Rationale
Backend Framework	FastAPI (Python 3.10+)	Asynchronous capabilities, automatic OpenAPI/Swagger documentation, speed, and Pydantic validation.
Web Server	Uvicorn	ASGI standard server providing high-throughput request handling.
Database Engine	SQLite	Zero-configuration file-based SQL database, ideal for lightweight

		embedded deployment.
Database ORM	SQLAlchemy 2.0	Provides advanced Object-Relational Mapping to interact with SQL tables via Python classes.
Frontend Layout	HTML5 & Vanilla CSS3	Ensures zero third-party framework overhead, rendering clean custom components, grids, and variables.
Frontend Scripting	Vanilla JavaScript (ES6)	Utilizes asynchronous Fetch API for server-client networking, dynamic DOM insertion, and state tracking.
Containerization	Docker (python:3.10-slim)	Enables isolated deployment inside standard Docker engines, packaging all necessary libraries.
CI/CD pipeline	GitHub Actions	Triggers automatic dependency installation, Python syntax checking, and Docker image builds upon code push.

3. System Features & Functional Modules

WealthWise is designed as a single workspace split into interactive sections. The primary functional modules are:

A. Real-Time Dashboard Summary

Located at the top of the interface, the dashboard provides a high-level summary of the user's finances. It updates instantly whenever transactions are added, edited, or deleted. It includes:

- **Total Spending Widget:** Aggregates the total spending from all saved transactions, styled in Indian Rupees (₹).
- **Transaction Counter Widget:** Displays the total count of expenses currently recorded in the system.
- **Top Category Widget:** Identifies the category with the highest cumulative spending and displays both the category name and its total amount.

B. Dynamic Expense Entry Form

The left sidebar contains a form allowing the creation of new transactions or editing of existing ones. The form incorporates validation constraints, such as requiring titles, enforcing strictly positive expense amounts (minimum ₹0.01), and a calendar selector for transaction dates.

C. Spending by Category Progress Indicators

The right-hand panel visualizes the breakdown of spending across predefined categories: Food, Shopping, Travel, Bills, Entertainment, and Other. Muted progress bars display the percentage breakdown of spending relative to the total expenditure, updating in real time.

D. Interactive Transaction Log

All transactions are displayed in a clean tabular view ordered chronologically (newest first). Each transaction card lists the date, title, category badge, amount, and notes. Row actions permit updating (reloading parameters into the entry form) or deleting (with confirmation alerts) transactions instantly.

4. Database Schema and Data Models

WealthWise stores and retrieves transactions from a single table named `expenses`. The schema is mapped in python using SQLAlchemy's Declarative Mapping system. This maps standard Python types to SQL fields.

Database Entity-Relationship Definition

Column Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, INDEX, AUTOINCREMENT	Unique identification number of the transaction.
title	VARCHAR(100)	INDEX, NOT NULL	Descriptive title of the expense (e.g., 'Weekly Groceries').
amount	DOUBLE / FLOAT	NOT NULL, > 0.00	Amount in Indian Rupees spent on this expense transaction.
category	VARCHAR(50)	INDEX, NOT NULL	The category class of the expense (Food, Travel, Bills, etc.).
date	DATE	NOT NULL	The calendar date on which the transaction occurred (YYYY-MM-DD).
notes	VARCHAR(500)	NULLABLE	Optional detailed logs or notes for the transaction.

ORM Model Definition in Python (SQLAlchemy)

The SQLAlchemy class schema implemented in 'backend/models.py' is as follows:

```
class Expense(Base):
    __tablename__ = "expenses"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, nullable=False, index=True)
    amount = Column(Float, nullable=False)
    category = Column(String, nullable=False, index=True)
    date = Column(Date, nullable=False)
```

```
notes = Column(String, nullable=True)
```

5. RESTful API Specification

The backend serves as a stateless RESTful service using FastAPI. Pydantic schemas enforce type safety and data integrity, ensuring client requests are checked prior to database operations.

Method	Endpoint	Request Body	Response Schema	Description
GET	/health	None	{"status": "healthy"}	Health check endpoint for monitor scripts.
POST	/expenses	ExpenseCreate	ExpenseResponse	Inserts a new expense transaction. Enforces amount > 0.
GET	/expenses	None	List[ExpenseResponse]	Retrieves list of all expenses ordered by date descending (newest first).
PUT	/expenses/{id}	ExpenseUpdate	ExpenseResponse	Modifies fields of an existing expense record by its ID.
DELETE	/expenses/{id}	None	ExpenseResponse	Removes a specific transaction from SQLite and returns its details.
GET	/summary	None	ExpenseSummaryResponse	Computes total sum, transaction counts, and grouped category spending.

6. User Interface Design and Frontend Implementation

The frontend is built using standard web technologies (HTML5, Vanilla CSS3, Vanilla JS) as a Single Page Application. The visual design is structured on a modern gray/white/blue theme featuring layout cards with responsive grid frameworks.

Key UI/UX Design Features

- **Visual Style:** Modern Outlined Aesthetic: Clean card containers, fine light borders, and explicit grid groupings instead of chaotic shadow effects.
- **Fonts & Styling:** Typography: The UI loads the Outfit sans-serif typeface from Google Fonts to construct a professional, scalable textual weight.
- **Asynchronous Updates:** Dynamic DOM Reloading: The Vanilla JS Fetch API reads from the backend `/expenses` and `/summary`, updating the table and widgets without full page reloads.
- **Categorized Visuals:** Category Badges: Distinct colorful badges highlight the transaction category in the table (e.g. green for Food, blue for Bills, violet for Shopping).
- **Intuitive Workflow:** Inline Edit Mode: Clicking 'Edit' pulls the transaction properties back into the main form, replaces the submit button style with an 'Update' button, and exposes a 'Cancel Edit' button.

Vanilla Javascript CRUD Lifecycle

The frontend JavaScript manages an internal state variable ``allExpenses``. When the user performs CRUD operations, the application behaves as follows:

- **On DOM Content Loaded:** Loads the summary API statistics and retrieves the full expense log list to draw the table.
- **On Form Submit:** Submits a POST request. If a hidden ID input is set, it fires a PUT request to update the record instead, resetting the form and updating the dashboard upon success.
- **On Delete Click:** Issues a DELETE request to `'/expenses/{id}'`. The client prompts for confirmation, then refreshes the summary stats and table.

7. Containerization & Deployment Configuration

To achieve rapid, reproducible software delivery, the application is fully containerized. A custom Dockerfile compiles and exposes the application server on a configurable port.

Dockerfile Structure

The project packages Python inside a lightweight base image, configures local directories, and mounts SQLite volumes:

```
FROM python:3.10-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY backend/ ./backend/
COPY frontend/ ./frontend/
RUN mkdir -p /app/data
ENV DATABASE_DIR=/app/data
VOLUME /app/data
EXPOSE 8000
CMD ["sh", "-c", "uvicorn backend.main:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

Deployment Blueprints (Render.com)

A render.yaml configuration file sits at the root of the project to allow one-click cloud service deployment. Render reads the file to spin up a Python Web Service, install requirements, and mount an external persistent disk at /app/data to preserve the SQLite database across restarts.

```
services:
- type: web
  name: wealthwise-expense-tracker
  env: python
  buildCommand: pip install -r requirements.txt
  startCommand: uvicorn backend.main:app --host 0.0.0.0 --port $PORT
  envVars:
    - key: DATABASE_DIR
      value: /app/data
  disk:
    name: expense-db-volume
    mountPath: /app/data
    sizeGB: 1
```

8. Continuous Integration & Pipeline Automation

To enforce software quality, a Continuous Integration (CI) pipeline is configured using GitHub Actions. The workflow automatically validates branch commits before they are merged into the principal branches.

GitHub Actions CI Pipeline configuration (ci.yml)

The workflow configuration file '.github/workflows/ci.yml' checks compilation integrity and docker builds:

```
name: CI Pipeline

on:
  push:
    branches: [ master, main ]
  pull_request:
    branches: [ master, main ]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository code
        uses: actions/checkout@v3
      - name: Set up Python 3.10
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Check code compilation (syntax check)
        run: |
          python -m compileall backend
      - name: Build Docker Image
        run: |
          docker build -t expense-tracker:latest .
```

9. Summary & Future Scope

WealthWise successfully implements a modern, single-page application structure for personal financial tracking. By utilizing FastAPI and SQLite, the application demonstrates excellent latency responses, zero memory leaks, and lightweight storage usage. Pydantic constraints ensure high database data validity, while Docker makes deployment simple.

Future Enhancements Plan

- **User Accounts & Authentication:** Develop user registration, encryption-backed login credentials, and session token storage.
- **Visual Charts & Graphs:** Construct line and bar graphs using lightweight libraries (e.g. Chart.js) to show weekly and monthly spending trends.
- **Multi-Currency Integration:** Include multi-currency support, permitting automatic exchange rate conversion to INR.
- **Report Export Features:** Incorporate expense export capabilities to standard CSV, PDF, or Excel sheets for advanced calculation.
- **Budget Limit Alerts:** Allow users to define strict maximum budget caps per category, alerting them when expenditures exceed 80% of the threshold.